4.3 PPT

4.3 Memory 体系：从当前会话走向长期记忆 v2

工作记忆会把每一步的实际结果整理成当前记录，**每一轮，程序都会把这些记录交给上下文装配器。**

这一节往后延伸一步：**任务结束以后，哪些信息还能在下次任务中继续使用？**

几天后用户新开会话，要加"退出登录"。上次的"正在准备补丁"没用了，但会话模块在哪、跑哪条测试命令、项目怎么处理过期会话，仍然有用。不保留，Agent 就得重搜，用户就得重说。

跨会话、跨任务保存并复用信息的能力，就是长期记忆。

本节路线：**选出值得存的信息 → 用 OKF 组织成文件，用 pi-okf 维护 → 用 pi-okf-memory 演示模型调工具存/取/撤 → 上下文丢失后怎么恢复。**

学完能说明三件事：**什么信息值得记；pi 怎样写成项目知识；下次用之前怎样判断是否仍适用。**

1. 区分短期记忆和长期记忆

1.1 当前任务需要什么，就放进短期记忆

回到登录会话修复。用户要求：恰好到期应返回无效。Agent 跑边界测试，预期 False、实际 True，准备把 `now <= expires_at` 改成 `now < expires_at`。

失败断言、补丁状态、待运行的测试都得留着——否则下一轮只收到一句"继续"，模型不知道代码改了没有。这就是**短期记忆**，工作记忆是其中按目标／进度／关键结果／未解决问题组织的部分。

"短期"说的是使用范围，不是只能放内存。

1.2 下次任务还需要什么，才考虑长期保存

修复结束后回头看，三条信息值得留：会话模块在 `app/session.py`、测试在 `tests/test_session.py`、项目约定"到期时刻必须无效"。

这三条信息，就可以进入长期记忆。

区别：

信息	当前任务中的用途	任务结束后的处理
补丁尚未应用	决定下一步要修改代码	任务结束后不作为当前进度复用
本次失败日志	定位这次错误	原文归档，必要时保存经验摘要与引用
会话模块路径	找到修改位置	作为项目线索保留，使用前核对
项目测试命令	验证当前修改	作为项目约定保留，配置变化后更新
到期边界规则	判断修复目标	若确属项目规则，可供以后任务复用

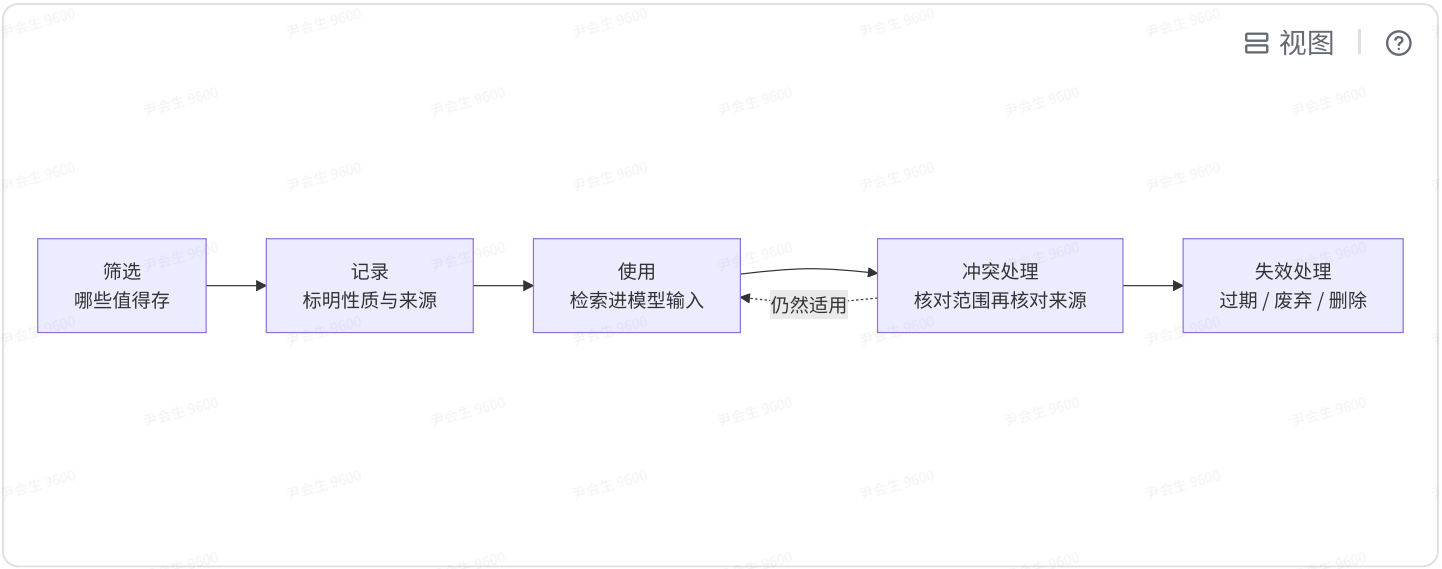
短期记忆服务于当前任务，任务结束就不再需要；长期记忆服务于以后的任务，所以必须能核对来源。

---

2. 长期记忆保存什么，怎样维护（生命周期）

两个重点：保存什么（哪些信息能进长期记忆、以什么性质保存）、怎样维护（保证内容后续仍然适用）。

将维护分成五个部分，它们分别承担下面的作用。



部分	为目标解决的问题	在登录案例中怎样体现
先判断值不值得记，再调用写入接口	确定长期记忆的保存范围，避免把临时进度和无用信息长期保留	保存会话模块位置、测试入口；“补丁准备半”留在当前任务状态里
可信度要说明信息是怎样得到的	确定保存时需要附带哪些来源信息，让以后读取的人或 Agent 判断怎样使用	“维护者要求到期无效”是约束；“测试通过”是有执行证据的事实；“可能与缓存有断
写入、检索、更新、删除分别解决什么	说明记忆从保存到使用、再到纠错和移除的基本操作	修复后保存知识；退出登录任务中检索；测化后更新；错误记录被删除
新旧记录冲突时，先比较范围与来源	规定更新时怎样选择有效内容，避免新消息直接覆盖可靠旧记录	新旧测试命令不一致，先确认项目和环境，前说明与执行结果
过期和删除不是同一件事	规定记录不再适用以后怎样处理，区分停止作为当前事实使用与彻底移除	旧版本的边界故障可以保留为历史经验；当经变化，旧入口应停用；误存的凭证则需要

2.1 保存什么：筛选后续任务仍然有用的信息

假设这次会话修复，产生了三句话。

1. 第一句：“会话逻辑在 app/session.py，对应测试在 tests/test_session.py。”
2. 第二句：“我猜退出登录的问题，可能和缓存有关。”这只是模型的推断。
3. 第三句：“这次为了排查问题，临时加了一个打印。”

写入之前，先问三件事：**以后哪类任务会用到它？它的来源能不能核对？它在什么条件下会失效？**

登录令牌、密码和完整用户请求载荷都不保存。

2.2 保存时记录什么：区分要求、事实和推断，附上来源

类别	登录会话项目中的例子	后续怎样使用
明确要求	项目维护者明确规定，不得通过延长有效期绕过边界问题	在适用范围内作为约束；修改操作还要经过执
验证事实	某个命令在指定环境与代码版本运行成功	作为有条件的事实；环境变化后重新验证
推断	模型判断某个缓存层可能影响退出登录	用来安排检查，不当作已经确认的原因
待确认信息	旧文档提到部署由 Supervisor 管理，尚未核对现网	暂不据此执行部署，先读取当前配置

记录时写明适用项目、来源、检查时间；涉及执行事实的补代码版本或文件哈希，让下次知道这条是在哪个现场验证的。

只存一句结论不够——必须说明它是要求、验证事实还是推断，下次才知道该遵守、该重核，还是只当线索。

2.3 保存后怎样使用和维护：让记忆持续服务后续任务

怎样让这些信息在后续任务中找得到、用得上，并在内容变化以后得到修正。

还是登录会话项目。修复完成以后，我们决定保留会话模块的位置和测试入口，因为以后修改登录相关功能，还需要找到代码并验证改动。

- **输入**是这次修复留下的两条结论；
- **动作**是把它们连同适用项目、来源和检查时间一起写入；
- **结果**是知识文件里多了两条带来源的记录。

几天后，用户要求增加退出登录功能。这个新任务需要知道：会话逻辑在哪里，修改以后怎样测试。

- **输入**是"退出登录"这个新任务加上当前项目标识；
- **动作**是程序按项目和任务检索相关记忆；
- **结果**是模块位置和测试入口两条记录进入这一轮的模型输入。

模型拿到这两条，先打开 `app/session.py` 看比较运算符，再打开测试文件看边界用例有没有覆盖 `now == expires_at`，然后才开始实现功能。

但项目会变化。假设会话测试已经从 `tests/test_session.py` 移到了 `tests/auth/test_session.py`，旧命令就不能继续作为当前入口。

如果某条记录本来就写错了，而且没有保留价值，就要删除。

更新和删除以前，都要确认记录属于当前项目，并定位到准确的记录 ID 或文件路径。搜索可以帮助找到候选，但不能因为两条内容相似，就直接覆盖或删除其中一条。

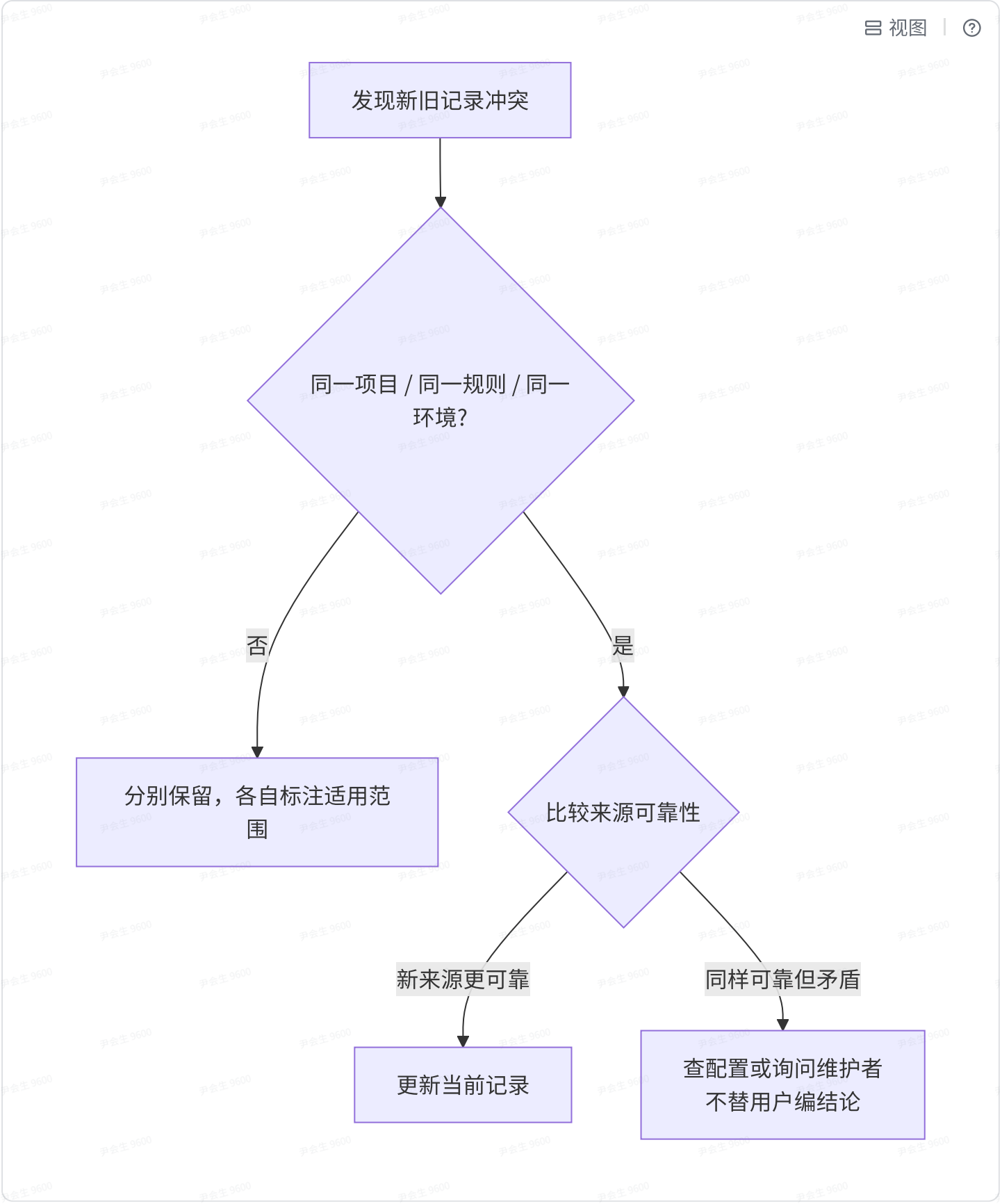
所以，这四个动作各管一段：**写入**，让有用信息留下来；**检索**，让它进入需要它的任务；**更新**，让它跟上项目变化；**删除**，让错误或不该保留的信息不再被读到。

2.4 更新依据是什么：新旧记录冲突时，先核对范围，再核对来源

旧记忆写着："会话测试使用 `pytest tests/test_session.py -q`。"新工程的说明要求使用 `python -m pytest tests/test_session.py -q`。

两条命令都可能有效，只是入口约定发生了变化。

不能因为新消息时间更晚，就自动删除旧记录。新消息可能只是模型建议，也可能针对另一个分支。



一句话：要求，要看谁有权提出；事实，要看证据是否适用。

2.5 失效以后怎样处理：过期、废弃和删除，解决不同问题

再看历史故障经验。我们保存这样一条："旧版本的到期判断使用了小于等于，导致到期时仍然有效；修复为小于以后，相关边界测试通过。"

过期＝不能再作为当前事实使用，不等于必须删除全部历史。删除＝移除记录，缓存和备份怎么清还要另行定义。

判断标准：不再适用之后，它还有没有历史价值？有就保留并标失效范围；没有就删除，防止继续进入后续任务的输入。

---

3. 用 OKF 组织项目知识，用 pi 插件维护和复用

3.1 为什么要约定统一格式

规则写在笔记、测试入口写在临时文件、经验藏在聊天记录里——下次任务开始时，Agent 不知道有哪些记录，也不知道哪条是当前规则、哪条是旧判断，只能重翻或问用户。

OKF（Open Knowledge Format）提供文件组织约定：正文用 Markdown，顶部用 YAML 记元数据，知识之间用链接连接。官方 v0.2 把知识包定义为 Markdown 目录树，每个概念文件有 YAML 头信息，`type` 始终必填，`index.md` 和 `log.md` 分管目录与更新历史。

课堂统一把知识包放在项目的 `~/.pi/agent/okf-memory` 下。这是我们的路径约定，OKF 不强制所有项目使用这个名字。

路径	内容
okf-memory/index.md	知识包入口
okf-memory/auth/index.md	登录与会话知识目录
okf-memory/auth/session.md	模块位置、当前会话规则
okf-memory/testing/session-tests.md	会话测试入口与覆盖要求
okf-memory/incidents/session-expiry.md	到期边界的历史修复经验
okf-memory/log.md	知识变更记录

3.2 为什么长期记忆不等于向量数据库

以下是 pi-okf-memory 实际生成的一份文件。看三处：顶部元数据、正文可复用信息、哪些仍需核对。

代码块

```
---
type: Fact
title: "登录会话模块：会话判断位置与到期边界规则"
description: "- 会话判断（会话有效性校验）的实现在 `app/session.py`。"
tags: [session, auth, 登录, 边界条件, 教学示例]
timestamp: 2026-10-06T07:48:49.407Z
source: session
---

# 位置

- 会话判断（会话有效性校验）的实现在 `app/session.py`。
- 使用者给出的记忆入口路径为 `../../app/session.py`（相对当前工作目录 `okf-memory/fact` 解析后未找到该文件，故实际路径以项目根目录下的 `app/session.py` 为准）。
- 使用前必须核对当前源码，不要凭记忆推断实现细节。

# 到期边界规则

- 项目（教学任务）规则：**`now == expires_at` 时，会话必须无效**。
- 即边界取闭区间失效：`now >= expires_at` → 无效；有效条件应写作 `now < expires_at`。
- 这条规则的适用范围仅限**本节登录示例**，不要推广到其他模块或全局。

# 核对清单

1. 打开 `app/session.py`，确认判断条件的比较运算符（`<` 还是 `<=`）。
2. 确认边界测试是否覆盖 `now == expires_at` 这一用例。
3. 如果模块路径或实现描述已经变化，核对后更新记忆；如果当前实现不满足“恰好到期时无效”的项目要求，先用边界测试确认问题，再修复代码。只有项目要求正式变更，才更新对应规则。
```

顶部：`type: Fact` 归为事实类；`title`、`description` 帮 Agent 判断相关性；`tags` 提供检索线索；`timestamp` 记时间；`source: session` 表示来自会话。

但这些字段只说明记录怎样被组织，不能证明内容已验证——`source: session` 没说是用户给的还是读源码确认的，时间戳也不是测试执行时间。

还有一处要留意：`description` 用确定语气写"实现在 `app/session.py`"，正文却承认相对路径没找到文件。摘要比正文更肯定，这是元数据与正文不一致的信号，以正文核对清单为准，把摘要当线索。

正文："位置"给源码入口，但明确写了相对路径没找到——所以 `app/session.py` 是先当待核对的定位线索，不能因为写进记忆就认定文件存在。"到期边界规则"存的是项目要求，并限定适用范围，防止被推广到其他模块。整份文件虽标 Fact，正文实际混合了不同性质：位置需核对，规则属要求，使用时要分别判断。

核对清单： 前两项把动作写得很具体——读源码、查比较条件、查边界测试。

回到"增加退出登录"：这份记忆先帮 Agent 找到会话代码和边界要求，再指导它读源码、查测试。它保存了可复用知识和核对步骤，但没有证明当前实现正确，也没有证明测试已通过。

3.3 安装与命令：终端和 pi 会话要分清楚

先在终端装插件：

代码块

```
pi install npm:pi-okf@0.4.0
```

装好以后，进入项目目录，启动 pi：

代码块

```
cd /path/to/your-project
pi
```

下面这些命令在 pi 会话内使用

命令	用途
/okf-init	读取当前项目的源码和说明，建立知识包。
/okf-update	根据仓库材料更新知识
/okf-update diff origin/main	根据相对指定 Git 引用的变化更新知识
/okf-update session	根据当前会话和工作更新知识
/okf-validate	检查知识包格式是否符合 OKF 规范。
/okf-compact	整理日志
/okf-compact all	整理概念、目录和日志

命令背后的知识编写仍然由模型完成，生成内容需要检查。

`/okf-validate` 检查格式，不负责证明登录功能正确。

3.4 演示一：建立知识，再从新会话读取

输入是当前项目文件夹。登录代码放在 `login_demo` 文件夹。

在 pi 会话里输入：

```
请基于当前源码、项目说明和实际测试记录建立知识。
重点记录会话模块位置、到期规则和测试入口。

不要修改业务源码，不保存凭证，不把推断写成验证事实。
没有执行证据的测试结果，明确标为待验证。

/okf-init
```

接着运行：

```
/okf-validate
```

现在关闭这个 pi 会话，再在同一个项目启动新会话，输入：

```
代码块
1  我要为 login_demo 增加退出登录功能，先不要修改代码。
2
3  先读取 ~/.pi/agent/okf-memory/index.md，
4  再读取与会话和测试有关的概念文件。
5
6  核对这些记录的来源和状态，并读取当前源码。
7
8  为我用中文输出需要检查的文件、适用约定和建议测试入口。
9  每项说明来自哪个知识文件，以及当前源码是否一致。
```

要看实际读取过程，不能只看回答里出现了熟悉的文件名。确认知识确实是从外部文件读进来的。

3.5 更新原文件，删除临时记录

维护者把入口统一为：

```
代码块
1  python -m pytest tests/test_session.py -q
```

先更新项目说明，再实际运行这个入口。确认结果以后，在同一个 pi 会话里说明：

```
代码块
1  维护者已调整会话测试入口，
2  当前项目说明和刚才的工具回执可供核对。
3
4  请更新 login_demo/ 中原有的测试概念文件。
5  保留原概念路径，替换当前入口，
6  并记录来源和验证范围。
7
8  旧命令若保留，只放在历史说明中。
```

然后执行：

```
/okf-update session
/okf-validate
```

核对三件事：

1. 当前入口已经换成 `python -m pytest ...`；
2. 来源和验证范围两栏有内容；
3. 旧命令出现在历史说明里，而不是当前入口。

三条都对，这次更新才算成功。之后 `/okf-validate` 没有报错，说明改动后的格式仍然合规。

如果使用 Git 差异更新，可以执行：

```
/okf-update diff origin/main
```

这条命令给的是变化线索——告诉你哪些文件动过。Agent 仍要读文件内容，不能只凭文件名编结论。

3.6 手动与自动修改记忆

手动可以通过斜杠命令实现：

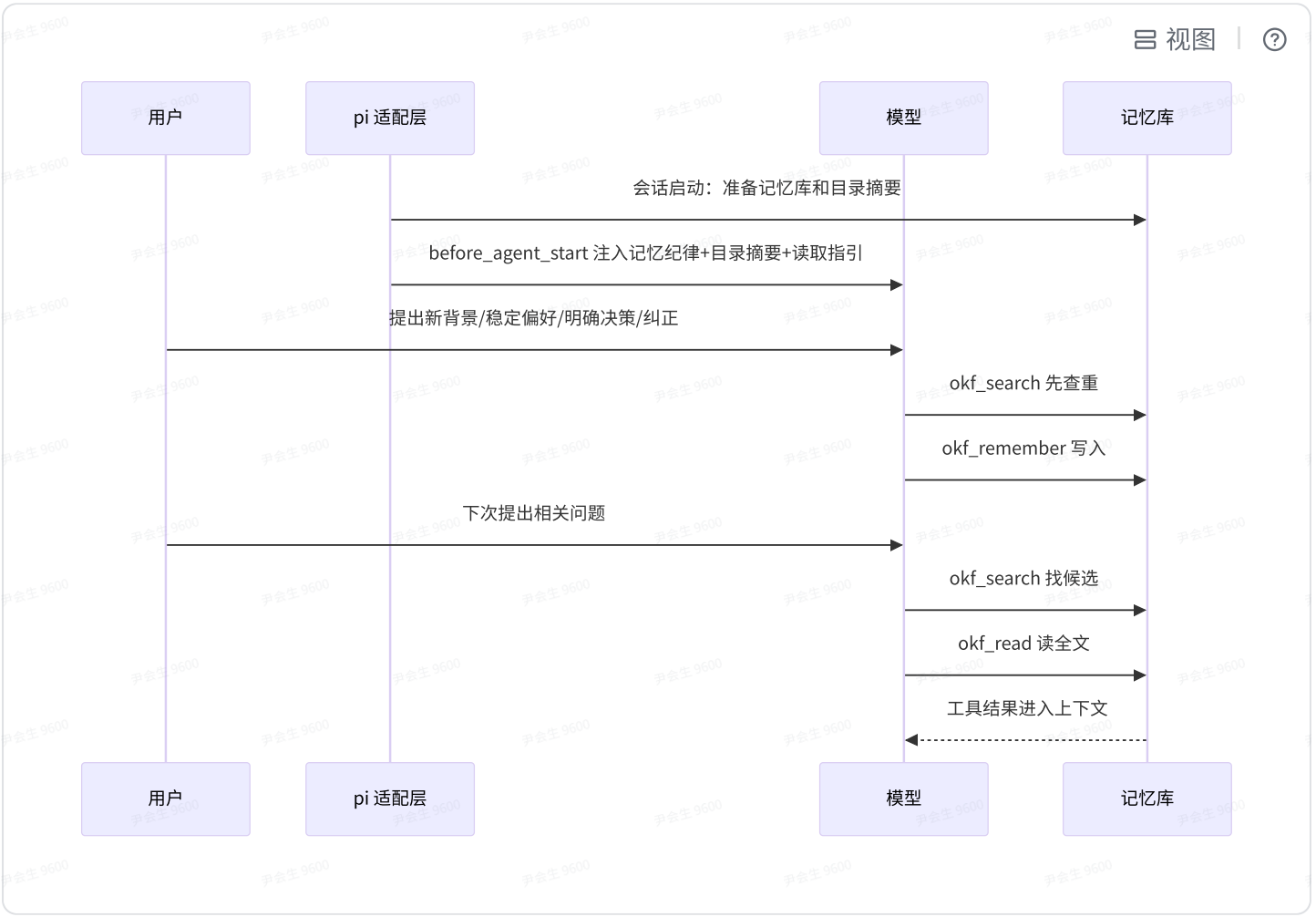
命令	操作
/okf 或 /okf status	查看根目录、概念数量和权重榜
/okf search 会话	查找会话相关记忆摘要
/okf graph	生成交互式图谱 HTML
/okf consolidate	执行权重衰减与归档整理
/okf help	查看命令帮助

日常对话由模型调用下面六个工具。

工具	输入与作用
okf_remember	用标题、类型和正文写入知识，返回实际概念 ID
okf_search	用查询词搜索，可按类型和标签过滤
okf_read	根据准确概念 ID 读取全文，并记录使用反馈
okf_forget	撤回指定概念；可选择删除文件
okf_feedback	对指定概念记录 select 或 skip 反馈
okf_graph	返回节点、链接和时间线数据

它们是工具名称，不是让用户在终端输入的命令。

"自动记住"具体怎样发生呢?



所以它能减少手动维护命令，但不是后台无条件保存每轮对话，也不是每次任务都确定性注入全部记忆。是否实际调用工具，要看执行记录。

看当前记录：

```
/okf graph
```

在 macOS 上，这条命令执行完会自动用系统的 `open` 命令打开生成的 HTML。如果你在 Windows 或 Linux 上，终端不会自动弹窗，这时候去看命令回执里给出的路径，手动打开那个文件。

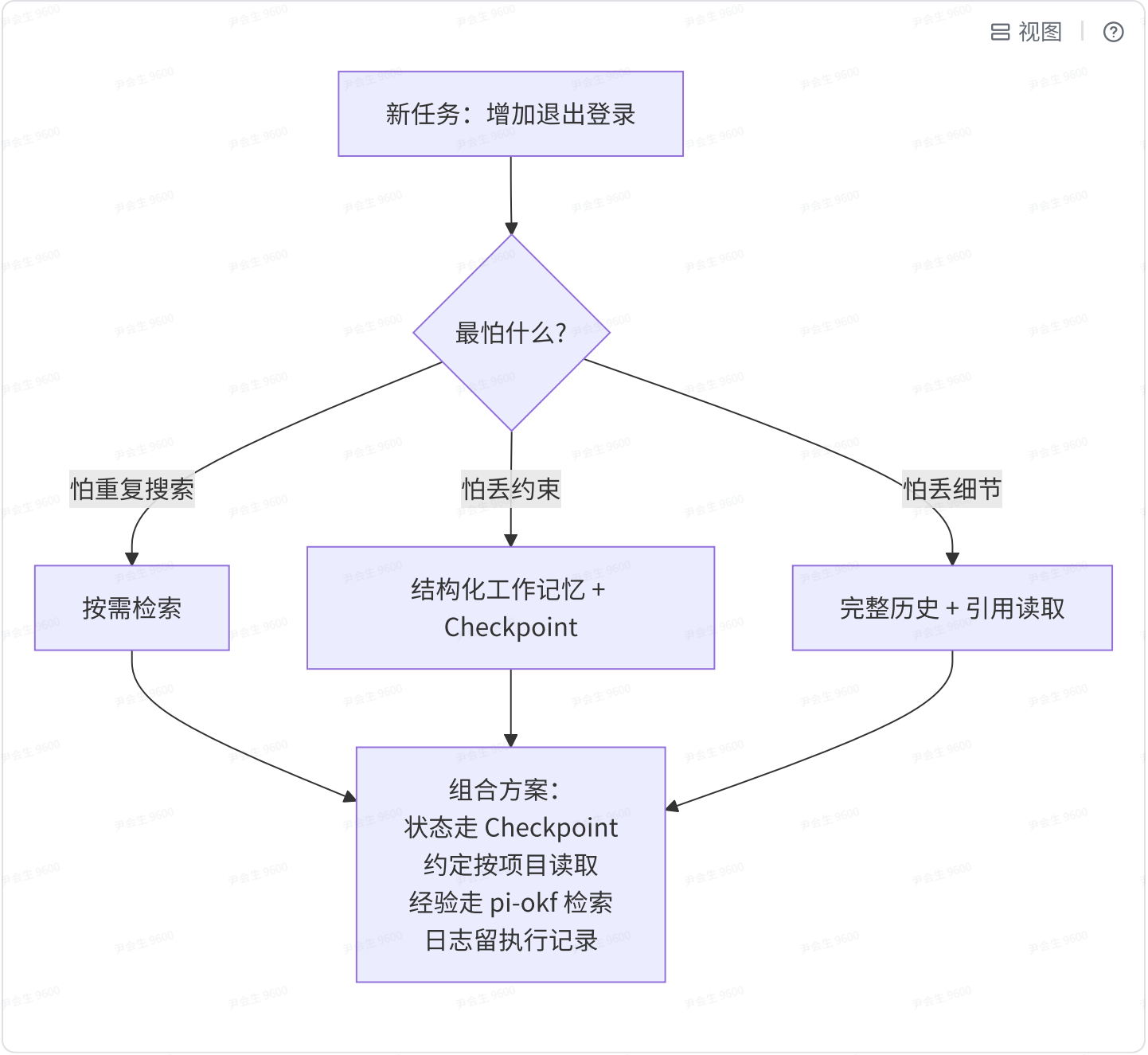
---

4. 记忆策略怎样比较：成功率、成本与延迟

4.1 四种策略，解决的问题不一样

我们仍然用“退出登录”这个任务，来做比较。

策略	可能减少的工作	需要检查的问题
无记忆	省去记忆服务接入	是否重复调查，是否缺少必要约定
完整历史	减少历史细节丢失	Token 增长、旧状态混入、窗口溢出
摘要记忆	减少每轮输入	摘要遗漏、来源丢失、状态被写错
按需读取 OKF	只发送相关项目知识	漏召回、冲突、陈旧记录、检索耗时



4.2 本项目先使用组合方案

对这个 Codebase Agent，我先采用组合方案：

- 目标、约束、进度 → 结构化工作记忆和 Checkpoint 精确恢复
- 必须遵守的项目约定 → 按项目读取，交给执行层检查
- 模块线索、历史故障经验 → pi-okf 按需检索
- 完整工具日志 → 留在执行记录，要看堆栈或补丁细节时再通过引用读取

小结：把有用的信息留下，把当前状态核对清楚

任务信息分成两种用途：目标、进度、执行回执帮完成这一次任务；模块位置、项目规则、经过核验的经验帮以后的任务少走弯路——不用再搜一遍目录，也不用再问用户测试命令是什么。

OKF 把后者组织成可读可改的文件；pi-okf 帮 pi 按项目证据创建和维护；pi-okf-memory 让模型在对话里通过工具存取记忆。新任务仍需实际读取并核对当前源码。

当前任务失去上下文时：摘要帮模型理解进度，Checkpoint 和工具回执帮程序判断动作是否完成，当前现场决定旧结果是否仍然有效。

下一节讲 RAG：在知识来源清楚的基础上，讨论资料变多后怎样检索、怎样选择证据、怎样检查回答是否受证据支持。